

Technical Write-up — GoComet Nova (Part 1)

1. Architecture



State lives in one place: a typed `PipelineState` threaded through `LangGraph`, checkpointed to `SQLite` after every node — not scattered across agent-local memory. Full reasoning for each architectural choice (why 3 agents, why this framework, why this model split) is in the PRD, §4–5.

2. Three Nastiest Failure Modes (found while testing, not hypothetical)

1. The "clean" shipment that wasn't. `shipment_1` — our supposedly clean fixture — failed validation because the rule set required `invoice_number` on every document, but Bills of Lading legitimately never carry one (it belongs on the Commercial Invoice). A naive validator applying every rule to every document manufactures false positives. **Fix:** document-type-aware rule filtering in the Validator — a `required` check is skipped for fields that don't structurally belong on that document type.

2. Cross-document unit mismatches that look like data-entry errors but aren't. Gross weight read `4,250 KG` on the BOL and `4.250 MT` on the Packing List — identical value, different units, and both naive string equality and a shallow LLM comparison flagged it as a mismatch, which would tank Straight-Through Processing for no reason. **Fix:** `_extract_numeric()` in the Validator parses value+unit, normalizes via `UNIT_CONVERSIONS` (KG/MT/LBS), and compares within a tolerance — reused as-is by the cross-validate node.

3. A crash mid-extraction leaves a shipment stuck forever, invisibly. If the process died while a large PDF was mid-extraction, the in-memory pipeline status vanished and the CG operator saw a shipment permanently "processing" with no way to know it was actually dead. **Fix implemented:** a custom `SQLiteBackedMemorySaver` persists `LangGraph`'s checkpoint after every node transition, so the state itself survives a restart (verified by kill-9'ing the server mid-run). **Fix not yet implemented, called out honestly:** nothing currently *reads* that checkpoint back in — a restart always begins a fresh run rather than resuming from the last completed node. That resume path (`graph.get_state()` + `graph.invoke(None, config)` on startup for any non-terminal shipment) is the top item in the production roadmap (PRD §8), not a solved problem today.

3. Observability — Tracing One Shipment at Scale

Today: a stable `shipment_id` is the join key across `shipments`, `documents`, and `validations`, and every LLM call's `tokens_used` / `processing_time_ms` / `model_used` is already captured per document (real columns, not proposed). What's missing for real production use: this data isn't yet aggregated into a live dashboard, and there's no distributed tracing across the extract → validate → route hop boundaries — you can currently reconstruct a shipment's full journey by querying three tables, but not by opening one trace view. **Production plan:** wrap each agent call in an OpenTelemetry span keyed by `shipment_id`, ship to a trace backend (or Langfuse, already present but commented out in `backend/requirements.txt`, ready to enable), and build one dashboard on top of data that already exists: STP rate, average confidence by supplier, error rate by rule, P95 extraction latency.

4. Cost — Measured, Not Estimated

Real numbers pulled from this build's own `documents` table across 20 extraction calls on Claude Haiku 4.5 (the default configured model, see PRD §5): **average 6,533 tokens per document, average 8.2s processing time per document**. A typical 4-document shipment (BOL + Invoice + Packing List + CoO) therefore costs roughly **26,000 tokens for extraction alone**, plus a smaller validation call per document and one routing call — validation and routing don't currently log token counts (a real gap, see roadmap). Using representative small-model list pricing for this tier (order of \$0.25–\$1 per million input tokens, a few dollars per million output — the exact figure moves with list pricing and isn't worth pretending to know to the cent), a full 4-document shipment lands in the **low single-digit cents** range end to end. **Where it blows up:** (a) a supplier attaching a 200-page catalog instead of a 1-page invoice — no page-count guard exists today, a real gap; (b) retry storms — capped today at 3 attempts per document (`max_retries=2`) plus one graph-level retry, but LangGraph's own `recursion_limit` is unset, which is a latent risk, not a theoretical one. **Control plan:** enforce a page-count ceiling at the trigger/upload boundary, wire the already-defined-but-unused `timeout_per_agent` setting into the actual LLM calls, and set an explicit `recursion_limit`.

5. Latency — Where Is the Slowest Hop?

Measured, not guessed: **extraction averages ~8.2s per document**, and it's genuinely parallelized across documents in a shipment (`asyncio.gather`), so a 4-document shipment's extraction wall-clock is close to the slowest single document, not the sum of all four. Validation and routing are comparatively fast (deterministic pre-gates run in milliseconds; the LLM calls for validation/routing are single, short completions). **The bottleneck is unambiguously the vision extraction hop.** To cut it further: (1) sniff for a clean PDF text layer before rendering to images/sending native PDF — if the text layer is good, a text-only prompt is both cheaper and faster than vision input; (2) the codebase already sends native PDF to Anthropic models rather than always rasterizing, which is the cheaper/faster path where it applies — extending "when do we even need the vision path" further is the next latency lever.

6. What I'd Do Differently With a Week

1. **Automated tests, first.** There is currently no pytest suite — the biggest single gap for calling this production-ready. A golden-set regression suite over the 14 sample shipments + 3 inbox scenarios (expected decision + expected flagged fields per scenario) would catch prompt-change regressions that manual testing currently has to catch by accident.
2. **Consume the checkpoints, not just write them** — implement actual crash-resume (§2, failure mode 3).
3. **Fix the two rule-enforcement gaps found while writing this doc:** the cross-validate node hardcodes a 1% tolerance instead of reading each customer's own `tolerance_pct`, and `present_on_all` rules are parsed from customer JSON but never enforced. Both are quick, high-trust-impact fixes.
4. **Tiered model routing** (Haiku default, Sonnet escalation on low confidence) instead of one fixed model per agent — see PRD §5.
5. **Move state to Postgres, keep SQLite's design.** The schema (shipments/documents/validations, checkpoint tables) doesn't need to change shape — SQLite just doesn't hold up under concurrent pipeline runs at real volume.